

Take-home assignment: "AI Decline Code Mapper" (up to 3 hours)

The goal is to evaluate your ability to design and implement a small AI agent that uses an LLM as a controlled component in a service architecture, not as a single black-box function.

Estimated time limit: **up to 3 hours** of focused work. Clean design and reasonable trade-offs are more important than exhaustive edge-case coverage.

Context

Imagine you are building an internal tool for payment integration engineers. When onboarding a new payment provider, one of the most tedious and error-prone tasks is **mapping the provider's error/decline codes** to our internal decline taxonomy.

A wrong mapping can cause:

- Failed transactions being retried indefinitely (if a terminal error is mapped as retryable).
- Legitimate fraud blocks being treated as system errors (triggering false alerts).
- Customer-initiated cancellations being reported as provider failures (skewing metrics).

You are given:

1. Internal decline taxonomy

Our platform uses the following decline categories. Every provider error must map to exactly one of these:

Category	Description	Retryable?
SYSTEM_MALFUNCTION	Internal system error, provider infrastructure failure, unexpected response	Yes (with backoff)
COMMON_DECLINE	Generic transaction decline by the bank or provider (insufficient funds, limit exceeded, card expired)	Depends on sub-code
ANTIFRAUD	Transaction blocked by fraud detection (provider-side or bank-side)	No
BADDATAPROVIDED	Invalid input data: wrong account number, invalid currency, malformed request fields	No (fix input first)
CANCELLEDBYCUSTOMER	Customer explicitly cancelled, abandoned, or refused the transaction	No
PROVIDER_LIMIT	Provider-side rate limit, daily/monthly volume cap, or account quota reached	Yes (after cooldown)
AUTHENTICATION_FAILURE	Invalid credentials, expired token, signature mismatch with the provider	No (fix config first)

2. Provider error code documentation

Below is the error code reference for a fictional provider **"QuickPay Global"**:

QuickPay Global – API Error Reference (v2.4)

All API responses include a ``result_code`` field (string) and a ``message`` field (human-readable).

=== Transaction Processing Errors ===

- QP-001 "Transaction declined"
The acquiring bank declined the transaction. No further details available.
- QP-002 "Insufficient balance"
The customer's account does not have enough funds.
- QP-003 "Card expired"
The payment card has passed its expiry date.
- QP-004 "Invalid card number"
The card number failed Luhn check or is not recognized by the network.
- QP-005 "Transaction not permitted"
The card issuer does not allow this type of transaction.
- QP-006 "Suspected fraud"
The transaction was flagged by the fraud detection system.
- QP-007 "3DS authentication failed"
The cardholder failed 3-D Secure verification.
- QP-008 "Do not honor"
The issuing bank declined without specifying a reason.
- QP-009 "Duplicate transaction"
A transaction with the same idempotency key was already processed.
- QP-010 "Amount limit exceeded"
The transaction exceeds the per-transaction or daily limit.

=== System / Connectivity Errors ===

- QP-100 "Internal server error"
An unexpected error occurred on QuickPay side.
- QP-101 "Service temporarily unavailable"
The payment gateway is undergoing maintenance or experiencing high load.
- QP-102 "Request timeout"
The upstream processor did not respond in time.
- QP-103 "Rate limit exceeded"
Too many API requests in the current time window. Retry after the period indicated in `Retry-After` header.

=== Validation Errors ===

- QP-200 "Missing required field: {field_name}"
A mandatory request field is missing.
- QP-201 "Invalid currency code"
The provided currency is not supported or not in ISO 4217 format.
- QP-202 "Invalid amount format"
Amount must be a positive number with up to 2 decimal places.
- QP-203 "Unsupported payment method"
The requested payment method is not enabled for this merchant.
- QP-204 "Invalid callback URL"
The provided notification URL is malformed or unreachable.

=== Authentication Errors ===

- QP-300 "Invalid API key"
The provided API key does not match any active merchant account.
- QP-301 "Signature verification failed"
The request signature (X-Sign header) does not match the computed value.

QP-302 "Token expired"
The OAuth bearer token has expired. Obtain a new token.

QP-303 "IP not whitelisted"
The request originated from an IP address not in the merchant's whitelist.

=== Customer Action Errors ===

QP-400 "Payment cancelled by user"
The customer clicked "Cancel" on the hosted payment page.

QP-401 "Session expired"
The customer did not complete the payment within the allowed time window.

QP-402 "Customer refused terms"
The customer declined the terms and conditions on the payment page.

3. Additional context — known ambiguities

In practice, some codes are genuinely ambiguous:

- **QP-005 "Transaction not permitted"** could be antifraud, a card restriction, or a merchant category code issue — the provider doesn't distinguish.
- **QP-008 "Do not honor"** is notoriously vague — issuing banks use it as a catch-all.
- **QP-009 "Duplicate transaction"** might be a legitimate idempotency guard (not an error at all) or a sign of a replay attack.
- **QP-401 "Session expired"** could be a customer abandonment or a system timeout, depending on how long the session window is.

Your task

Implement a small service (CLI tool, HTTP API, or simple web UI — your choice) that, given a provider's error code documentation (as text):

1. **Parses the error codes** into a structured list of `(code, name, description)` tuples.
2. **Maps each code** to one of our internal decline categories, producing:
 - The selected category.
 - A confidence level: `high`, `medium`, or `low`.
 - A short reasoning explaining the mapping choice.
3. **Flags ambiguous codes** that need human review, with an explanation of why the mapping is uncertain and what additional information would help resolve it.
4. **Suggests a retry strategy** for each code:
 - `no_retry` — terminal error, do not retry.
 - `retrywithbackoff` — transient error, retry with exponential backoff.
 - `retryafterfix` — retryable only after the merchant/customer fixes something.
 - `no_action` — not a real error (e.g., idempotency guard).
5. **Returns the result** as a machine-readable JSON array.

LLM/agent requirements

Mandatory points:

- **Use the LLM as part of an agent architecture**, not as a single "magic" function:
- Explicitly separate stages: parsing the raw documentation → enriching with taxonomy context → mapping each code → validating/reviewing the output.
- This can be implemented as a custom orchestrator or via any agent framework.
- **Return the result in a machine-readable format:**

```
{
  "provider": "string",
  "version": "string",
  "mappings": [
    {
      "provider_code": "string",
      "provider_message": "string",
      "internal_category": "string",
      "confidence": "high|medium|low",
      "reasoning": "string",
      "retry_strategy": "no_retry|retry_with_backoff|retry_after_fix|no_action",
      "needs_human_review": false,
      "review_reason": "string|null"
    }
  ],
  "summary": {
    "total_codes": 0,
    "high_confidence": 0,
    "needs_review": 0,
    "unmapped": 0
  }
}
```

- **Handle typical model errors:**
- Invalid JSON.
- Hallucinated categories (the model invents a category not in the taxonomy).
- Inconsistent confidence (says "high" but the reasoning describes ambiguity).
- Missing codes (the model skips some codes from the input).
- **Show a recovery strategy:**
- For example, validate that every input code appears in the output; if not, re-prompt for the missing ones.
- Validate that `internal_category` is one of the 7 allowed values; if not, re-prompt with the taxonomy.
- Cross-check that `needs_human_review` is `true` whenever `confidence` is `low`.

Technical conditions

- Programming language and stack — your choice.
 - You may use:
 - Any public LLM API (OpenAI, Anthropic, Gemini, etc.) or a local model.
 - Any LLM/agent libraries/frameworks (LangChain, Semantic Kernel, etc.).
 - Important:
 - Move configuration (API keys, endpoints) to external settings/environment variables.
 - Keep the solution compact and readable — no need for heavy enterprise boilerplate.
-

What to submit

1. A link to the repository (GitHub/GitLab/Bitbucket, etc.) with your code.
 2. A **README** briefly describing:
 - How to run the project (minimal steps).
 - How the agent is structured at a high level (1-2 paragraphs or a simple step-by-step description).
 - How you tested the behavior (e.g., run it on the QuickPay documentation above and at least one other provider you invent or find).
 3. (Optional, but appreciated) A short note on trade-offs:
 - What you simplified to fit into ~3 hours.
 - What you would do differently with more time (e.g., batch processing of multiple providers, interactive disambiguation UI, integration with a decline code database).
-

Mini example of expected behavior (non-canonical)

Input (excerpt):

```
QP-006  "Suspected fraud"
        The transaction was flagged by the fraud detection system.
```

Example of a possible (schematic) answer (single entry):

```
{
  "provider_code": "QP-006",
  "provider_message": "Suspected fraud",
  "internal_category": "ANTIFRAUD",
  "confidence": "high",
  "reasoning": "The provider explicitly states the transaction was flagged by fraud detection. This directly maps to",
  "retry_strategy": "no_retry",
  "needs_human_review": false,
  "review_reason": null
}
```

Example of an ambiguous code:

```
{
  "provider_code": "QP-005",
  "provider_message": "Transaction not permitted",
  "internal_category": "COMMON_DECLINE",
  "confidence": "low",
  "reasoning": "The provider gives no detail on why the transaction is not permitted. It could be a card restriction",
  "retry_strategy": "no_retry",
  "needs_human_review": true,
  "review_reason": "Ambiguous: could be ANTIFRAUD or BAD_DATA_PROVIDED depending on issuer behavior. Recommend checki"
}
```